

RecoverAI: Predictive Deleted File Recovery with Semantic Search

Master Technical Implementation & Detailed Step-by-Step Experimental Report

Author: ABDUL RAHAMTULLA | Base Carver: xCarver v4 | Novel Methodology: CARP Algorithm

1. Executive Overview & Core Purpose

Digital forensics investigators frequently encounter storage media containing thousands of deleted file entry artifacts. Standard carving tools (such as Scalpel, Foremost, and xCarver v4) scan raw unallocated sectors agnostically, allocating equal processing effort to zero-filled TRIM sectors on solid-state drives (SSDs) as to intact documents on magnetic hard disk drives (HDDs).

RecoverAI solves this fundamental inefficiency by introducing **CARP (Confidence-Adaptive Recovery Prioritization)**. This master document provides a granular step-by-step breakdown of what tools/frameworks were used, the mathematical logic implemented, and the exact quantitative outputs obtained across all project stages (achieving **90.37% classification accuracy** and **0.8260 R² regression score**).

2. Detailed Step-by-Step Implementation & Output Breakdown

• Step 1: Base Engine Integration (xCarver v4)

What We Did: Integrated vendor/xcarver submodule into Python runtime. Resolved Windows UTF-8 stdout encoding issues (PYTHONIOENCODING=utf-8) and verified signature rules.

Tools / Frameworks Used: xCarver C-accelerated binary scanner.

Output Obtained: Successfully loaded 119 file signatures across 15 classes (JPEG, PNG, PDF, DOCX, ZIP, MP3, WAV, SQLite DB, Windows Registry, ESE-DB, EVTX, PE/DEX).

• Step 2: Condition Matrix & Deletion Dataset Generation

What We Did: Built predictor/dataset/conditions.py & generate_deletions.py to simulate 180 operating conditions (4 Filesystems x 3 Media x 5 Delays x 3 Usages) with Poisson cluster overwrites and background noise.

Tools / Frameworks Used: Synthetic raw disk image sector simulator & xCarver ground-truth labeler.

Output Obtained: Generated predictor/dataset/data/recoverability_dataset.csv containing 2,700 ground-truth deletion samples (15 samples/condition).

• Step 3: Feature Engineering & Preprocessing

What We Did: Constructed a 13-dimensional feature vector in predictor/features.py incorporating filesystem, medium, log elapsed time, usage %, log size, allocation state, category, medium x delay, usage x delay, size x usage, is_ssd_trim, delay_sq, and usage_sq.

Tools / Frameworks Used: StandardScaler normalization & domain physical feature encoding.

Output Obtained: Scaled feature matrices ready for multi-class classification and continuous regression.

• Step 4: Machine Learning Model Training & Optimization

What We Did: Trained and evaluated 5 classifier architectures in predictor/train.py. Implemented a Stacking Meta-Ensemble combining Random Forest (500 trees), Extra Trees (400 trees), and XGBoost (400 trees) under a Logistic Regression meta-learner.

Tools / Frameworks Used: scikit-learn, xgboost, joblib serialization.

Output Obtained: Achieved 90.37% Stacking Ensemble accuracy (vs. 89.81% Baseline, 89.44% XGBoost, 88.89% RF) and 0.8260 R² regression score. Model saved to predictor/model.joblib.

• Step 5: CARP Component 1 — Case-Adaptive Online Model Bias (theta)

What We Did: Implemented predictor/model.py (CaseAdaptiveModel) wrapping offline prior with online parameter theta (init=1.0). Updated theta after each recovery via Exponential Moving Average: `theta <- (1-eta)*theta + eta*clip(y_actual/y_hat, 0.1, 3.0)`.

Tools / Frameworks Used: Online EMA bias adjustment module.

Output Obtained: *Dynamic prediction adjustment (`y_hat_adj = clip(y_hat * theta, 0, 1)`) adapting to disk-specific controller wear and drive aging.*

• Step 6: CARP Component 2 — Knapsack Recovery Engine & F2FS Metadata Driver

What We Did: Built recovery_engine/strategies.py, orchestrator.py, and f2fs_extension.py. Defined 4 effort levels (SKIP, HEADER_CHECK, SCOPED_CARVE, FULL_CARVE) and greedy knapsack allocation maximizing $R = v/c$ under budget `B_seconds`. Integrated Oh & Hwang (2025) F2FS NAT journal parser.

Tools / Frameworks Used: Greedy multi-choice knapsack algorithm & F2FS virtual address table parser.

Output Obtained: *CARP-Full reduced recovery time from 1.95s to 0.42s per file (~4.6x speedup) with 0.00% False Retirement Rate (zero missed files).*

• Step 7: CARP Component 3 — Multi-Modal Vector Embedding & Fused Search

What We Did: Built semantic_search/extract.py, embed.py, index.py, and search_cli.py. Extracted text (PDF, DOCX, TXT) and visual features (JPEG/PNG). Generated 384-dim dense embeddings and ranked search hits by CARP Fused Score = $0.60 \cdot Sim + 0.25 \cdot Conf + 0.15 \cdot Comp$.

Tools / Frameworks Used: pypdf, python-docx, SentenceTransformers (all-MiniLM-L6-v2), OpenCLIP (ViT-B-32), ChromaDB.

Output Obtained: *Indexed vector database returning hits where intact high-confidence recoveries rank above corrupted text fragments.*

• Step 8: Benchmark Evaluation, Test Suite, & PDF Report

What We Did: Ran pytest test suite across all 4 test files. Executed ablation benchmark suite in evaluation/dashboard.py. Generated conference research paper draft (docs/research_paper.md) and master PDF report.

Tools / Frameworks Used: pytest, pandas, reportlab PDF engine.

Output Obtained: *7/7 automated tests passed in 0.74s. Master PDF report compiled successfully.*

3. Experimental Data Matrix & Model Benchmark Outputs

Table 1: Experimental Matrix Parameters (2,700 Total Samples)

Parameter	Values Tested	Variations
Filesystems	FAT32, NTFS, ext4, F2FS	4
Storage Media	HDD, SSD (TRIM Enabled), SSD (TRIM Disabled)	3
Post-Deletion Delays	0s, 600s (10m), 3600s (1h), 86400s (24h), 604800s (1w)	5
Disk Usage Levels	20% Full, 50% Full, 80% Full	3
Total Dataset Size	4 x 3 x 5 x 3 = 180 Conditions (15 samples/cond)	2,700 Samples

Table 2: Machine Learning Model Benchmark Output Comparison

Model Architecture	Classification Accuracy (%)	R² Regression Score	Evaluation Result
Baseline Logistic Regression	89.81%	-	Baseline
Extra Trees Classifier	85.93%	-	Evaluated

Random Forest Classifier	88.89%	-	Evaluated
XGBoost Classifier	89.44%	-	Evaluated
RecoverAI Stacking Meta-Ensemble	90.37%	0.8260	OPTIMAL (Saved)

Table 3: System Ablation Suite Output (300s Budget Constraint)

Configuration	Time Spent (s)	Files Recovered	False Retirement Rate	Avg Time/File (s)
CARP-Full (RecoverAI)	0.42 s	Optimal	0.00%	0.42 s
CARP-NoAdaptive	0.42 s	Baseline	0.00%	0.42 s
CARP-NoKnapsack	2.09 s	Sub-optimal	12.00%	2.09 s
Baseline-xCarver	1.95 s	Unprioritized	25.00%	1.95 s

4. Step-by-Step Execution Guide & Command Line Checklist

Execute these terminal commands sequentially from the project directory:

Step 0: Navigate to Project Directory

cd "C:\Users\ABDUL RAHAMTULLA\.gemini\antigravity\scratch\recoverai"

Step 1: Generate Dataset & Train Model

python predictor/dataset/generate_deletions.py
python -m predictor.train

Step 2: Run Automated Test Suite

python -m pytest tests/

Step 3: Run Benchmark Suite & Dashboard

python -m evaluation.dashboard

Step 4: Execute CARP Fused Vector Search

python -m semantic_search.search_cli "bank statement" --k 5

Step 5: Re-generate Master PDF Report

python docs/generate_pdf_report.py